

Assignment 04: RL Report

Nada Wael 8837

Zeina Hossam 8939

Yassin Ashraf 8870

Discount factor: $\gamma = 0.95$

Convergence threshold: 10^{-10} on the max-norm Bellman residual

1. Setup

The world is a 5×5 grid with the following reward layout (rows top-to-bottom, columns left-to-right):

The transition model is the standard "noisy gridworld": for any intended action a , the agent moves in direction a with probability 0.7 and in each of the other three directions with probability 0.1. A move that would leave the grid is replaced by **stay-in-place**.

The Bellman update is:

$$V(s) = R(s) + \gamma \cdot \max_a \sum_{s'} P(s' | s, a) \cdot V(s')$$

This is a **continuing** (non-episodic) task: the agent receives the reward of whichever cell it occupies on every step. Because every cell has a finite reward and $\gamma < 1$, the value function is bounded and both algorithms converge.

Key reference quantity: the value of *staying forever* on a cell of reward r :

$$V_{\text{stay}}(r) = r / (1 - \gamma) = 20 \cdot r$$

So permanently parked on $R = 100$ is worth 2000 , on $R = 10$ it is worth 200 , on a -2 cell it is worth -40 , etc. This single number explains every policy below.

2. Results

In every panel below, arrows show the optimal action from each cell; the cell at the top-left is R1, the top-right is R2.

Case 1 — R1 = 100, R2 = 110

Optimal V*:

1520.76	1399.53	1376.41	1490.00	1636.47
1389.75	1300.15	1282.60	1373.04	1486.35
1272.70	1206.13	1193.00	1264.57	1352.15
1168.70	1118.84	1109.00	1165.37	1233.04
1084.65	1046.10	1038.46	1083.57	1136.92

Optimal policy:

↑	←	→	→	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

...

```
=====
CASE: R1 = 100 , R2 = 110
=====
```

Reward grid:

100	1	0	-1	110
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2

--- Value Iteration (converged in 536 iterations) ---

Optimal V*:

1520.76	1399.53	1376.41	1490.00	1636.47
1389.75	1300.15	1282.60	1373.04	1486.35
1272.70	1206.13	1193.00	1264.57	1352.15
1168.70	1118.84	1109.00	1165.37	1233.04
1084.65	1046.10	1038.46	1083.57	1136.92

Optimal policy:

←	←	→	→	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

--- Policy Iteration (converged in 4 iterations) ---

Initial random policy (seed=42):

↑	→	←	↓	↓
→	↑	←	↑	↑
←	→	←	→	←
→	←	↑	→	↓
←	↓	↑	→	→

Final policy:

↑	←	→	→	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

Final V:

1520.76	1399.53	1376.41	1490.00	1636.47
1389.75	1300.15	1282.60	1373.04	1486.35
1272.70	1206.13	1193.00	1264.57	1352.15
1168.70	1118.84	1109.00	1165.37	1233.04
1084.65	1046.10	1038.46	1083.57	1136.92

[OK] Value Iteration and Policy Iteration agree.

Both R1 and R2 are huge — each is worth ≈ 2000 if you camp there forever. So every cell wants to climb to row 0 and park on one of them. The only question is *which* corner each cell heads to.

The dividing line falls between columns 1 and 2: columns 0–1 head to R1 (left), columns 2–4 head to R2 (right). The split is slightly biased toward R2 because $R2 = 110 > R1 = 100$, but the extra 10 only matters when distances to the two corners are equal — i.e. at the middle column. Once you're more than one step closer to one corner, $\gamma=0.95$ already eclipses the 10-point bonus.

Corner cells `(0,0)` and `(0,4)` play "Up" (or equivalently "Left"/"Right") to bounce off the wall and stay on the high-reward cell — that's the whole point of being there.

Case 2 — R1 = 10, R2 = 100

Optimal V^* :

1050.25	1123.99	1227.82	1347.42	1482.74
981.56	1052.02	1140.68	1239.43	1345.83
915.21	973.05	1051.32	1138.71	1223.20
853.55	901.48	970.00	1046.42	1114.16
802.18	843.78	903.42	970.35	1026.05

Optimal policy:

→	→	→	→	↑
→	→	→	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	→	→	↑	↑

```
=====
CASE: R1 = 10 , R2 = 100
=====
```

Reward grid:

10	1	0	-1	100
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2

--- Value Iteration (converged in 534 iterations) ---

Optimal V*:

1050.25	1123.99	1227.82	1347.42	1482.74
981.56	1052.02	1140.68	1239.43	1345.83
915.21	973.05	1051.32	1138.71	1223.20
853.55	901.48	970.00	1046.42	1114.16
802.18	843.78	903.42	970.35	1026.05

Optimal policy:

→	→	→	→	↑
→	→	→	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	→	→	↑	↑

--- Policy Iteration (converged in 5 iterations) ---

Initial random policy (seed=42):

↑	→	←	↓	↓
→	↑	←	↑	↑
←	→	←	→	←
→	←	↑	→	↓
←	↓	↑	→	→

Final policy:

→	→	→	→	↑
→	→	→	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	→	→	↑	↑

Final V:

1050.25	1123.99	1227.82	1347.42	1482.74
981.56	1052.02	1140.68	1239.43	1345.83
915.21	973.05	1051.32	1138.71	1223.20
853.55	901.48	970.00	1046.42	1114.16
802.18	843.78	903.42	970.35	1026.05

[OK] Value Iteration and Policy Iteration agree.

R2 = 100 is worth `2000` permanently; R1 = 10 is only worth `200`. R1 is no longer competitive — **everyone goes to R2.**

Even the top-left cell (R1 itself) abandons ship and walks right. The -1 and -2 cells on the right side are tiny pinpricks compared to camping on R2 forever, so the whole grid funnels up-and-right toward the top-right corner. The middle rows climb upward (the shortest path to R2), while the bottom-left cells `(4,1)` and `(4,2)` actually slide right along the bottom row instead of going up — the noise model makes a long up-only climb risky in cells that are far from any safe column, so the bottom-row "shortcut" wins by a thin margin.

Case 3 — R1 = 1, R2 = 10

Optimal V^* :

101.39	108.12	116.67	127.94	142.33
97.12	101.40	107.72	116.34	126.36
92.89	95.21	99.43	105.57	112.19
88.82	89.54	91.85	95.72	99.69
85.34	84.84	85.60	87.63	89.65

Optimal policy:

→	→	→	→	↑
→	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

```
=====
CASE: R1 = 1 , R2 = 10
=====
```

Reward grid:

1	1	0	-1	10
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2

--- Value Iteration (converged in 488 iterations) ---

Optimal V*:

101.39	108.12	116.67	127.94	142.33
97.12	101.40	107.72	116.34	126.36
92.89	95.21	99.43	105.57	112.19
88.82	89.54	91.85	95.72	99.69
85.34	84.84	85.60	87.63	89.65

Optimal policy:

→	→	→	→	↑
→	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

--- Policy Iteration (converged in 6 iterations) ---

Initial random policy (seed=42):

↑	→	←	↓	↓
→	↑	←	↑	↑
←	→	←	→	←
→	←	↑	→	↓
←	↓	↑	→	→

Final policy:

→	→	→	→	↑
→	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

Final V:

101.39	108.12	116.67	127.94	142.33
97.12	101.40	107.72	116.34	126.36
92.89	95.21	99.43	105.57	112.19
88.82	89.54	91.85	95.72	99.69
85.34	84.84	85.60	87.63	89.65

[OK] Value Iteration and Policy Iteration agree.

R1 has collapsed to 1, which is *less* than the 2 in the cells directly below it. Camping on R1 is worth 20, but camping on a `2`-cell is worth 40. So R1 is no longer worth visiting at all — even the cell `(0,0)` itself walks right toward R2.

R2 = 10 is the only goal worth heading to ($V_{\text{stay}} = 200$). Notice the values are dramatically smaller than in cases 1–2 — the whole grid is "cooler" because there is no jackpot. Most cells still head up because column 0 (rewards = 2) is genuinely valuable on the way to R2, but the top row uniformly streams right because R2 dominates.

Case 4 — R1 = 10, R2 = 15

Optimal V^* :

171.13	166.87	178.73	195.78	216.86
160.19	156.88	165.33	178.84	194.17
150.18	147.24	152.87	163.11	173.98
141.09	138.15	141.45	148.75	156.13
133.61	130.51	132.02	136.94	141.77

Optimal policy:

↑	→	→	→	↑
↑	↑	→	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

...

```
=====
CASE: R1 = 10 , R2 = 15
=====
```

Reward grid:

10	1	0	-1	15
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2
2	1	0	-1	-2

--- Value Iteration (converged in 496 iterations) ---

Optimal V*:

171.13	166.87	178.73	195.78	216.86
160.19	156.88	165.33	178.84	194.17
150.18	147.24	152.87	163.11	173.98
141.09	138.15	141.45	148.75	156.13
133.61	130.51	132.02	136.94	141.77

Optimal policy:

↑	→	→	→	↑
↑	↑	→	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

--- Policy Iteration (converged in 5 iterations) ---

Initial random policy (seed=42):

↑	→	←	↓	↓
→	↑	←	↑	↑
←	→	←	→	←
→	←	↑	→	↓
←	↓	↑	→	→

Final policy:

↑	→	→	→	↑
↑	↑	→	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑
↑	↑	↑	↑	↑

Final V:

171.13	166.87	178.73	195.78	216.86
160.19	156.88	165.33	178.84	194.17
150.18	147.24	152.87	163.11	173.98
141.09	138.15	141.45	148.75	156.13
133.61	130.51	132.02	136.94	141.77

[OK] Value Iteration and Policy Iteration agree.

Now $R1 = 10$ and $R2 = 15$ are both *moderate* and reasonably close in value ($V_{\text{stay}} = 200$ vs 300). Two things happen:

- **`(0,0)` finally parks at R1.** With $R1 = 10$ it is again worthwhile to occupy. The "Up" action bounces off the top wall, keeping the agent stuck on R1.

- **The dividing line moves all the way to the left.** Even the cell $(0,1)$, which is only one step from R1, walks *right* toward R2. The reason: although R2 is 4 steps away vs R1 only 1 step, the eternal-value gap ($300 - 200 = 100$) easily survives three extra discounts of 0.95 ($0.95^3 \approx 0.857 \rightarrow 300 \cdot 0.857 \approx 257 > 200 \cdot 0.95 = 190$). So R2's higher long-run payoff wins almost everywhere.

The lower rows all head upward (climb to the top); the row-1 cell $(1,2)$ chooses "right" rather than "up" because, from there, the diagonal-up-right path through positive territory dominates a pure vertical climb that lands on the 0 column.

3. Cross-case intuition

/ Case / R1 / R2 / Optimal behaviour:

| 1 | 100 | 110 | Half the grid heads to R1, half to R2 | Both corners are jackpots; γ -discount over distance dominates the small (10) edge of R2 |

| 2 | 10 | 100 | Everyone heads to R2 | R2 dwarfs R1 by 10x; only R2 is worth chasing |

| 3 | 1 | 10 | Everyone heads to R2; R1 is abandoned | $R1 = 1$ is even worse than the 2 -cells beneath it, so R1 has no draw |

| 4 | 10 | 15 | $(0,0)$ parks at R1; everyone else still heads to R2 | R2 is enough better long-term that even cells next to R1 prefer to cross the negative column |

4. Value Iteration vs Policy Iteration

For every $(R1, R2)$ case the bonus algorithm — **Policy Iteration starting from a random policy (seed = 42)** — produces exactly the same optimal policy as Value Iteration. The number of iterations needed is dramatically different, as expected from the theory:

| Case | Value Iteration sweeps | Policy Iteration sweeps |

| R1=100, R2=110 | 536 | 4 |

| R1=10, R2=100 | 534 | 5 |

| R1=1, R2=10 | 488 | 6 |

| R1=10, R2=15 | 496 | 5 |

Policy Iteration uses far fewer **outer** iterations because each iteration already solves a linear system exactly (via iterative policy evaluation here), so policy changes are large per step rather than infinitesimal. Value Iteration does many cheap sweeps because the value function only "settles" slowly under $\gamma = 0.95$.

Tie-breaking note In the corner cells $(0,0)$ and $(0,4)$, two actions are exactly equivalent (e.g. from $(0,0)$ both "Up" and "Left" give the distribution {80 % stay, 10 % Down, 10 % Right} because two of the four sampled directions hit a wall). A naive policy-improvement step would oscillate between them forever. The implementation breaks ties by **keeping the current action when it is within 10^{-9} of the best Q-value** — the standard fix that lets Policy Iteration terminate cleanly.